

Schedulers in PyTorch

Welcome to this exploration of learning rate schedulers in PyTorch! As you have learned in the lectures, managing learning rates is vital to optimizing model training, and PyTorch provides various tools to streamline this process. Learning rate schedulers are essential for dynamically adjusting learning rates according to the training phase, helping to facilitate faster convergence and improve generalization.

In this notebook, you will dive into the mechanics of different learning rate schedulers. Specifically, you will:

- Explore how schedulers like `StepLR`, `CosineAnnealingLR`, and `ReduceLROnPlateau` adjust the learning rate to maintain training efficiency.
- Use these schedulers within a typical training loop.
- Learn to visualize training dynamics, demonstrating the impact of schedulers on model performance compared to a constant learning rate.

By the end of this notebook, you will have practical experience in applying learning rate schedulers, enhancing your understanding of their role in model training.

```
import sys
import time
import warnings

# Redirect stderr to a black hole to catch other potential messages
class BlackHole:
    def write(self, message):
        pass
    def flush(self):
        pass
sys.stderr = BlackHole()

# Ignore Python-level UserWarnings
warnings.filterwarnings("ignore", category=UserWarning)

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

import helper_utils

helper_utils.set_seed(42)

# # Check device
if torch.cuda.is_available():
```

```

device = torch.device("cuda")
print(f"Using device: CUDA")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
    print(f"Using device: MPS (Apple Silicon GPU)")
else:
    device = torch.device("cpu")
    print(f"Using device: CPU")

```

Using device: CUDA

A deeper dive into the training process

In this section you will make a deeper analysis of the experiments we have done in the previous lab. You will use a simple model (`SimpleCNN`) and the CIFAR-10 dataset, but now we will inspect the training process more closely by plotting the **training loss** and **validation accuracy** curves.

This will help use understand how the model learns over time and how different learning rates affect the training dynamics.

In the following cell we find `SimpleCNN` model definition, and `evaluate_epoch` function that you can use to evaluate the model on the validation set *after each epoch*.

`train_and_evaluate` function is provided to train the model for a specified number of epochs and evaluate it during training.

`get_data_loaders` and `train_epoch` functions are provided from `helper_utils` module. They are used to load the CIFAR-10 dataset and train the model for one epoch, respectively.

```

class SimpleCNN(nn.Module):
    """A simple Convolutional Neural Network (CNN) architecture.

    This class defines a two-layer CNN with max pooling, dropout, and
    fully connected layers, suitable for basic image classification
    tasks.
    """
    def __init__(self):
        """Initializes the layers of the neural network."""
        # Initialize the parent nn.Module class
        super(SimpleCNN, self).__init__()
        # First convolutional layer (3 input channels, 16 output
        channels, 3x3 kernel)
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        # Second convolutional layer (16 input channels, 32 output
        channels, 3x3 kernel)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        # Max pooling layer with a 2x2 window and stride of 2
        self.pool = nn.MaxPool2d(2, 2)
        # First fully connected (linear) layer

```

```

        self.fc1 = nn.Linear(32 * 8 * 8, 64)
        # Second fully connected (linear) layer, serving as the output
layer
        self.fc2 = nn.Linear(64, 10)
        # Dropout layer for regularization
        self.dropout = nn.Dropout(p=0.4)

    def forward(self, x):
        """Defines the forward pass of the network.

        Args:
            x (torch.Tensor): The input tensor of shape (batch_size,
3, height, width).

        Returns:
            torch.Tensor: The output logits from the network.
        """
        # Apply first convolution, ReLU activation, and max pooling
        x = self.pool(F.relu(self.conv1(x)))
        # Apply second convolution, ReLU activation, and max pooling
        x = self.pool(F.relu(self.conv2(x)))
        # Flatten the feature maps for the fully connected layers
        x = x.view(-1, 32 * 8 * 8)
        # Apply the first fully connected layer with ReLU activation
        x = F.relu(self.fc1(x))
        # Apply dropout for regularization
        x = self.dropout(x)
        # Apply the final output layer
        x = self.fc2(x)
        return x

def train_and_evaluate(learning_rate, device, n_epochs=25,
batch_size=128, p_bar=None):
    """Orchestrates the training and evaluation of a model for a given
configuration.

    This function handles the end-to-end workflow: setting a random
seed,
    initializing the model, optimizer, loss function, and dataloaders,
and then
    running the main training loop.

    Args:
        learning_rate (float): The learning rate for the optimizer.
        device: The device (e.g., 'cuda' or 'cpu') for training and
evaluation.
        n_epochs (int, optional): The number of training epochs.
Defaults to 25.
        batch_size (int, optional): The batch size for dataloaders.

```

Defaults to 128.

p_bar (optional): An existing progress bar handler. Defaults to None.

Returns:

dict: A dictionary containing the training and validation history (loss and accuracy).

"""

Set the random seed for reproducibility

helper_utils.set_seed(42)

Initialize the model and move it to the specified device

model = SimpleCNN().to(device)

Define the optimizer with the specified learning rate

optimizer = optim.Adam(model.parameters(), lr=learning_rate)

Define the loss function

loss_fn = nn.CrossEntropyLoss()

Prepare the training and validation dataloaders

train_loader, val_loader = helper_utils.get_dataset_dataloaders(
 batch_size=batch_size
)

Call the main training loop to train the model and get the history

history = helper_utils.train_model(
 model=model,
 train_dataloader=train_loader,
 val_dataloader=val_loader,
 optimizer=optimizer,
 loss_fcn=loss_fn,
 device=device,
 n_epochs=n_epochs,
 p_bar=p_bar
)

Return the collected training history

return history

Now the model is trained for several epochs for each of the specified learning rates. The results are stored in `training_curves`, and the curves (train loss and validation accuracy) are plotted for each learning rate.

Different learning rates to be analyzed

learning_rates = [0.0002, 0.001, 0.005] *# Small, medium, and large learning rates*

```

training_curves = []
n_epochs = 25
batch_size = 128

p_bar = helper_utils.get_p_bar(n_epochs)

# Get the total number of learning rates to check against the index
num_learning_rates = len(learning_rates)

# Train and evaluate the model for each learning rate
for i, lr in enumerate(learning_rates):
    print(f"\nTraining with learning rate: {lr}\n")
    history = train_and_evaluate(learning_rate=lr, n_epochs=n_epochs,
batch_size=batch_size, device=device, p_bar=p_bar)
    training_curves.append(history)
    # Only reset the progress bar if it's NOT the last iteration
    if i < num_learning_rates - 1:
        p_bar.reset()

{"model_id": "a86b863c6fc944cd9377d3d401003a66", "version_major": 2, "version_minor": 0}

{"model_id": "c1bb0261999c43d7bbf5c7898473acdd", "version_major": 2, "version_minor": 0}

```

Training with learning rate: 0.0002

```

{"model_id": "915e7a0b088448018e336eb75eeaa9f2", "version_major": 2, "version_minor": 0}

{"model_id": "e82be8322e5d4192ba53a34340bc5bc3", "version_major": 2, "version_minor": 0}

{"model_id": "fa0eca09e6c84a039872ced7eb404569", "version_major": 2, "version_minor": 0}

{"model_id": "9be0d64ed02743cd8bd7103a008c010c", "version_major": 2, "version_minor": 0}

{"model_id": "1865b339af394017b0a93019979c7b2d", "version_major": 2, "version_minor": 0}

```

At epoch 5: Training loss: 1.7604, Training accuracy: 0.3705
At epoch 5: Validation loss: 1.6633, Validation accuracy: 0.4130

```

{"model_id": "2f19d4b8f2c44064b9dfd71d83a63bda", "version_major": 2, "version_minor": 0}

{"model_id": "170435bb17c74040b7ac4f5d0cbb1010", "version_major": 2, "version_minor": 0}

```

```
{"model_id": "5c168dbfe9eb4d14bacc21b6f120e33c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c19f5ead991848c6ae2f40cda74a392e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1ca2eef66da841119c73b17b6453605a", "version_major": 2, "version_minor": 0}
```

At epoch 10: Training loss: 1.5486, Training accuracy: 0.4519
At epoch 10: Validation loss: 1.5006, Validation accuracy: 0.4720

```
{"model_id": "2d69fc8850fc4bcfa6ce538ed556c68b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9845eddf3fe74725829d4a558d167059", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "57106444751541c58d46375360a94d49", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "33d82048aa4e4c6996e730945da58a9f", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "87125523565445a2b849d21ada420203", "version_major": 2, "version_minor": 0}
```

At epoch 15: Training loss: 1.4553, Training accuracy: 0.4723
At epoch 15: Validation loss: 1.4378, Validation accuracy: 0.4905

```
{"model_id": "135e187f39b147c7af396e268eb48e5a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5968b14e89c044ae8a5c67031b45c6f2", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "95f46ba79e9c43be804e93d4a2e7901c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "24c72a8285074461881a391cbf8336ba", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2d793d45c9734968b639cea7e5a6d7cc", "version_major": 2, "version_minor": 0}
```

At epoch 20: Training loss: 1.3774, Training accuracy: 0.4986
At epoch 20: Validation loss: 1.3793, Validation accuracy: 0.5130

```
{"model_id": "4b392f2170e446b9a330bd5c7a12755e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b1ceb8aa9f804c1f8eb434bb063e02ef", "version_major": 2, "version_minor": 0}
```

```
{"model_id":"a5b990dc55b447c7b9b5f689b472acf3","version_major":2,"version_minor":0}
```

```
{"model_id":"c1a742edc2fb430e965a43ab72911f94","version_major":2,"version_minor":0}
```

```
{"model_id":"4034fce81c4a4fa79b8a930014039b1a","version_major":2,"version_minor":0}
```

At epoch 25: Training loss: 1.3216, Training accuracy: 0.5186

At epoch 25: Validation loss: 1.3435, Validation accuracy: 0.5220

Training complete

Training with learning rate: 0.001

```
{"model_id":"69752d3f0df54f18a06a9c0443718cdf","version_major":2,"version_minor":0}
```

```
{"model_id":"89cceff304974e93839ad10bffffef1e2","version_major":2,"version_minor":0}
```

```
{"model_id":"99b745d787c94340aac2e07ecba636a3","version_major":2,"version_minor":0}
```

```
{"model_id":"e3d41fc8e40e4ba99b80276fbc38366a","version_major":2,"version_minor":0}
```

```
{"model_id":"4b5c14a6f28c44f792a8f884bebd0627","version_major":2,"version_minor":0}
```

At epoch 5: Training loss: 1.4890, Training accuracy: 0.4655

At epoch 5: Validation loss: 1.4216, Validation accuracy: 0.4880

```
{"model_id":"12bda71120c642f0abfaf65f857ea0ab","version_major":2,"version_minor":0}
```

```
{"model_id":"480cf9c5c2a94c489eb1bcf421711ea3","version_major":2,"version_minor":0}
```

```
{"model_id":"5664b8a2be7f4dd780c7d59ae5b360a8","version_major":2,"version_minor":0}
```

```
{"model_id":"cd369238263e4a55a5099f5584ed4937","version_major":2,"version_minor":0}
```

```
{"model_id":"0a9129e114a6438092e78d2f40c28ac5","version_major":2,"version_minor":0}
```

At epoch 10: Training loss: 1.2904, Training accuracy: 0.5377

At epoch 10: Validation loss: 1.2997, Validation accuracy: 0.5350

```
{"model_id": "78f7e8b71f5d4ac883816487f919cebc", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "43f93505c31448cd962a0395b0523e5e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cd681f73c99f4dabba45e80f623847f4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9c60fd9bfebb4386a74a6e566110f824", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f543d283157146e99e439068dc0dd66d", "version_major": 2, "version_minor": 0}
```

At epoch 15: Training loss: 1.1575, Training accuracy: 0.5759

At epoch 15: Validation loss: 1.2196, Validation accuracy: 0.5645

```
{"model_id": "9cc100a1622d4ba1a3ed6eaf02d77e42", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e9d1dc62ca7b41b2915e0b4485fbe054", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1f6376793d55450296b4291f8b0f901a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ad92852f27484cdf815fc6366db8bacd", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8ad6d6b6d77940868aad05eca428a5b6", "version_major": 2, "version_minor": 0}
```

At epoch 20: Training loss: 1.0248, Training accuracy: 0.6276

At epoch 20: Validation loss: 1.1890, Validation accuracy: 0.5800

```
{"model_id": "2ff3cf6ab8924c83aa8cf32863a1357c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f9b455e486a7448f92c776df518bcdd4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "0e5364d079764cc3b554820468df8526", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9d291c27166d4380bc84b7ba307183ef", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "004fb10e5442410a9514d6850716f876", "version_major": 2, "version_minor": 0}
```

At epoch 25: Training loss: 0.9041, Training accuracy: 0.6676

At epoch 25: Validation loss: 1.2110, Validation accuracy: 0.5885

Training complete

Training with learning rate: 0.005

```
{"model_id": "06fe797150644437bdce48608f3b00af", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "54ceabab90de4a709551256816571a24", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c360b31bb26d4211b5b39f4a29c97149", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "77c5f55b41624fc2a2994ccb7baf676", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6b21bf82f25a4aab89261b2ce322ce59", "version_major": 2, "version_minor": 0}
```

At epoch 5: Training loss: 1.4156, Training accuracy: 0.4791

At epoch 5: Validation loss: 1.3789, Validation accuracy: 0.4785

```
{"model_id": "855793bb55714dabb51b2f101c18e761", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "07bbececa2ff486a930179f2422bc9ae", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b453bafb1dea40bfa875a72c50aec5b5", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1f7d22524905422db9b6d078ef65f47a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "81e1bfc338b04c0d8a6059b8c8ecb0ae", "version_major": 2, "version_minor": 0}
```

At epoch 10: Training loss: 1.1888, Training accuracy: 0.5651

At epoch 10: Validation loss: 1.2673, Validation accuracy: 0.5555

```
{"model_id": "4e620af1ccdb4a2099673c02404fefea", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "89e4fc6ac7a442cd8a711639ce2b013b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d886856c8b4a4849a579ef3f6926718f", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d8874f223f9246e2859ec5b92a0e3fc8", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "82961140b38845768c23790d9affd20e", "version_major": 2, "version_minor": 0}
```

```
At epoch 15: Training loss: 1.0018, Training accuracy: 0.6254  
At epoch 15: Validation loss: 1.3083, Validation accuracy: 0.5610
```

```
{"model_id": "eeddabe8b0be45588cd5a20c993123aa", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "cc53d50280ab4fdbb6d179c13453662d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "87302c6e952f4bcfa495cd145659e556", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f6fa683b90f44bfe8c4305dcc013f8d6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6c111d753d184a168c742f41d93cb101", "version_major": 2, "version_minor": 0}
```

```
At epoch 20: Training loss: 0.8938, Training accuracy: 0.6561  
At epoch 20: Validation loss: 1.3363, Validation accuracy: 0.5640
```

```
{"model_id": "9c3ec4b5b6f24d81b53c1a3c8fc3c8da", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "79698ab4784f4ab7b7bd88fe833b975e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d4c678a580b4468ea73a67757dd9d191", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9ddd961eeeb0459397b97b4c17dafc55", "version_major": 2, "version_minor": 0}
```

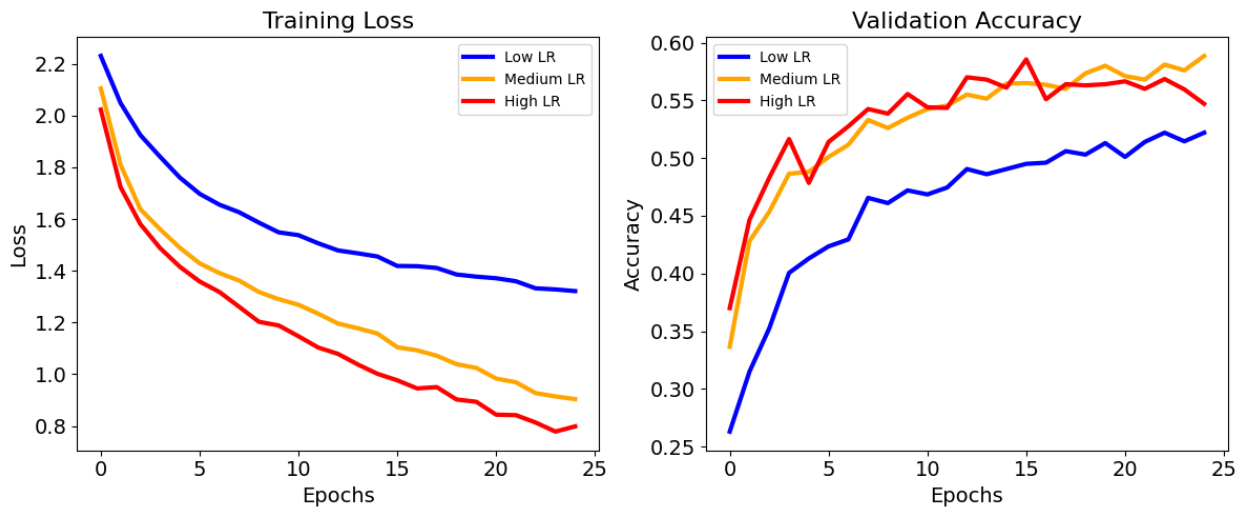
```
{"model_id": "1d13076a387b458a9e8187ec9e5def5c", "version_major": 2, "version_minor": 0}
```

```
At epoch 25: Training loss: 0.7988, Training accuracy: 0.6950  
At epoch 25: Validation loss: 1.4847, Validation accuracy: 0.5470  
Training complete
```

```
colors = ['blue', 'orange', 'red']  
labels = ['Low LR', 'Medium LR', 'High LR']
```

```
helper_utils.plot_learning_curves(colors, labels, training_curves)
```

```
# Each color corresponds to a different learning rate: blue for low,  
orange for medium, and red for high.
```



Learning Rate Analysis

Each color represents a different learning rate: blue for low, orange for medium, and red for high.

- **Low Learning Rate:** The validation accuracy improves gradually and may require a significant amount of time (many epochs) to reach its peak.
- **Medium Learning Rate:** The validation accuracy increases quickly, and by the end of the 25 epochs, it is outperforming the other two models.
- **High Learning Rate:** Initially, validation accuracy is the highest during the first few epochs but eventually levels off, indicating instability where the model struggles to improve with such a high learning rate.

Among these options, the medium learning rate strikes the best balance between learning speed and generalization. Could you achieve a better model by combining these learning rates—starting with a high rate for the first few epochs, then transitioning to medium, and finally to low?

Schedulers

In the previous section, you explored how the choice of learning rate impacts model performance. A high learning rate can lead to instability, while a low one might slow down training or yield suboptimal outcomes. Learning rate schedulers offer a solution by automatically adjusting the learning rate throughout training, initially setting it high for rapid learning and then reducing it to enhance convergence and generalization.

In this lab, you'll investigate three different schedulers. Let's start with the `StepLR` scheduler, which lowers the learning rate by a specified factor at regular intervals—specifically, after a set number of epochs. You'll repeat the training process from before, but this time you'll employ a scheduler that decreases the learning rate by a factor of 0.2 every 10 epochs.

```

helper_utils.set_seed(42)

# Initialize the model, optimizer, loss function, and dataloaders
model = SimpleCNN().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.005) # start with a
high learning rate

```

Schedulers are built into the PyTorch `torch.optim.lr_scheduler` module and can be easily integrated into your training loop. You typically initialize a scheduler with the optimizer and parameters that define how the learning rate changes over time.

The `StepLR` scheduler, for example:

- Takes the optimizer as input.
- Requires `step_size`, the number of epochs before reducing the learning rate.
- Uses a `gamma` factor, the factor by which the learning rate is reduced.

```

scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10,
gamma=0.2) # reduce the learning rate by 20% it's prior value

```

The training process is similar to the previous one, but the learning rate is adjusted by the scheduler after each epoch using the `scheduler.step()` method. The current learning rate can be accessed using `scheduler.get_last_lr()`.

The model is trained for several epochs with the `StepLR` scheduler, and the training curves are plotted to compare the results with the previous experiments for the medium learning rate.

```

loss_fn = nn.CrossEntropyLoss()

train_loader, val_loader =
helper_utils.get_dataset_dataloaders(batch_size=batch_size)

history_LR = {
    "train_loss": [],
    "train_acc": [],
    "val_loss": [],
    "val_acc": [],
    "lr": [],
}

pbar = helper_utils.NestedProgressBar(
    total_epochs=n_epochs,
    total_batches=len(train_loader),
    epoch_message_freq=5,
    mode="train",
)

for epoch in range(n_epochs):
    pbar.update_epoch(epoch+1)

```

```

    # Train the model for one epoch
    train_loss, train_acc = helper_utils.train_epoch(model,
train_loader, optimizer, loss_fn, device, pbar)

    # Evaluate the model on the validation set
    val_loss, val_acc = helper_utils.evaluate_epoch(model, val_loader,
loss_fn, device)

    # Get the current learning rate BEFORE stepping the scheduler.
    # This captures the LR that was just used for the training epoch
    above.
    current_lr = scheduler.get_last_lr()[0]

    # Step the scheduler (updates the LR for the NEXT epoch)
    scheduler.step()

    pbar.maybe_log_epoch(epoch=epoch+1, message=f"At epoch {epoch+1}:
Training loss: {train_loss:.4f}, Training accuracy: {train_acc:.4f},
LR: {current_lr:.6f}")

    pbar.maybe_log_epoch(epoch=epoch+1, message=f"At epoch {epoch+1}:
Validation loss: {val_loss:.4f}, Validation accuracy: {val_acc:.4f}")

    history_LR["train_loss"].append(train_loss)
    history_LR["train_acc"].append(train_acc)
    history_LR["val_loss"].append(val_loss)
    history_LR["val_acc"].append(val_acc)
    history_LR["lr"].append(current_lr)

pbar.close('Training complete with StepLR scheduler')

{"model_id": "b139aef932ad4ef6974b168149da9cad", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "65b62201ffb74d30a9eb2cb84a935f63", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "d800cfce83554b83acca7f6176861427", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "d44da1931c554daf8c98956efc43d8e2", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "acb2767a7dc4489387b659d895f115f7", "version_major": 2, "vers
ion_minor": 0}

{"model_id": "863c3e180b074b0290417f339d040e6b", "version_major": 2, "vers
ion_minor": 0}

```

```
{"model_id": "05994cc7fede40d2a003e7ea6faa8aca", "version_major": 2, "version_minor": 0}
```

At epoch 5: Training loss: 1.4212, Training accuracy: 0.4748, LR: 0.005000

At epoch 5: Validation loss: 1.3891, Validation accuracy: 0.4910

```
{"model_id": "95254cdd1ac04c60bc4b3ce2d9c98767", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d15551990928444eacf17fc21f73f129", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "01dec3a2f0f043bb8f3965814be7b766", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "30c511136c1042d8aeedff2a01705f15", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "185ff3f3837f4145870415d97bbf5f28", "version_major": 2, "version_minor": 0}
```

At epoch 10: Training loss: 1.1795, Training accuracy: 0.5686, LR: 0.005000

At epoch 10: Validation loss: 1.2928, Validation accuracy: 0.5480

```
{"model_id": "71d9d1ac5d6b40e0a0f5b95b12a632c4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b1846789f48245d2a5578fb0ef927d75", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "194d6d8a14e3425e96753ff31bac54cd", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e91dc5ed95894ef093b9a497431e6ccf", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "99e9c456be9d4b00b10a22624bb20ea5", "version_major": 2, "version_minor": 0}
```

At epoch 15: Training loss: 0.9722, Training accuracy: 0.6442, LR: 0.001000

At epoch 15: Validation loss: 1.2553, Validation accuracy: 0.5730

```
{"model_id": "64498861305c4e139e2c14186856de82", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6307aaee65654907a2a1b6a54d6f1bf5", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e582ebb5dbeb4e698f13cb1c58714791", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "a1bef1047c8e4f7c82a07dd56d90d2fc", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c25651ceb30247d190b1d845d7fc1173", "version_major": 2, "version_minor": 0}
```

At epoch 20: Training loss: 0.9071, Training accuracy: 0.6639, LR: 0.001000

At epoch 20: Validation loss: 1.2794, Validation accuracy: 0.5675

```
{"model_id": "e6eef4c328354dc5a5dc9f0bcae4697b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d6092a0a1d0a4658a2486e11789de211", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ccf6efb11e284c5a9e60781d1379c94e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2151d3628b584d11aafab7f4d9b2c622", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3cc46ab3218c429b83194e642d543ab9", "version_major": 2, "version_minor": 0}
```

At epoch 25: Training loss: 0.8416, Training accuracy: 0.6855, LR: 0.000200

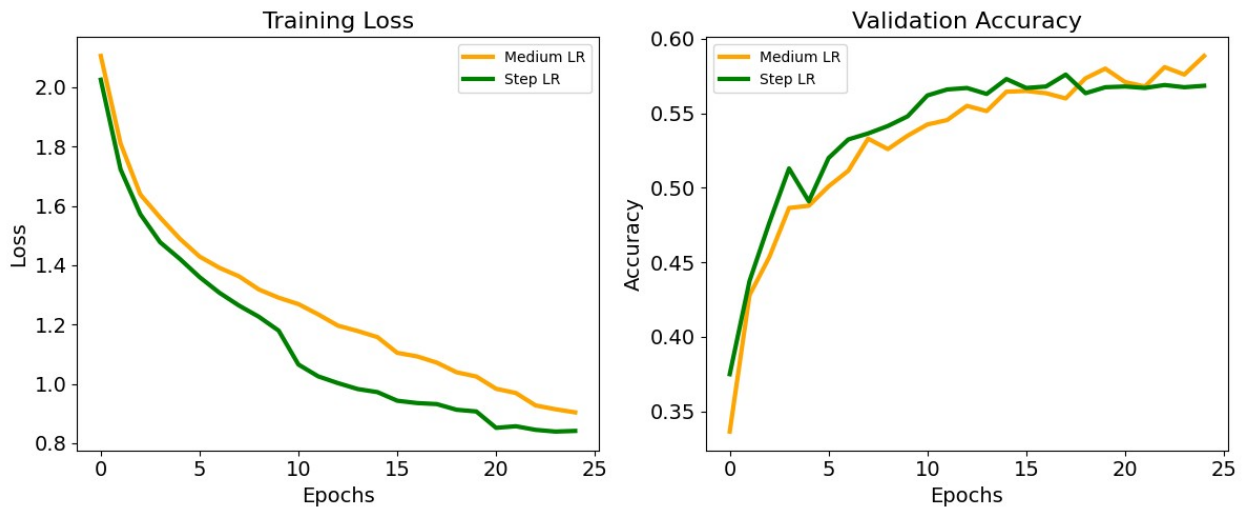
At epoch 25: Validation loss: 1.2992, Validation accuracy: 0.5685

Training complete with StepLR scheduler

```
idx = 1  
history_constant = training_curves[idx]
```

```
colors = ['orange', 'green']  
labels = ['Medium LR', 'Step LR']  
histories = [history_constant, history_LR]
```

```
helper_utils.plot_learning_curves(colors, labels, histories)
```



Analysis of Medium LR and StepLR Scheduler

The plots show a comparison between training with a constant learning rate (orange) and using a StepLR scheduler (green). With StepLR, the training loss is significantly lower than with a fixed learning rate. You can also notice a few dips at the epoch 10 and 20 when the learning rate changes. Although the validation accuracy starts off stronger, both models eventually converge to similar performance levels at the last epoch. Now, let's examine two more schedulers to compare their effectiveness.

Other Schedulers

There are many other schedulers available in PyTorch, such as `CosineAnnealingLR`, and `ReduceLROnPlateau`, each with its own strategy for adjusting the learning rate. You can explore these options in the [PyTorch documentation](#).

Below `train_and_evaluate_with_scheduler` function extends the previous `train_and_evaluate` function to include a learning rate scheduler.

```
def train_and_evaluate_with_scheduler(model, optimizer, scheduler,
device, n_epochs=25, batch_size=128):
    """Trains and evaluates a model using a learning rate scheduler.

    Args:
        model: The neural network model to be trained.
        optimizer: The optimization algorithm.
        scheduler: The learning rate scheduler.
        device: The computing device ('cuda' or 'cpu') to run the
training on.
        n_epochs: The total number of training epochs.
        batch_size: The number of samples per batch in the data
loaders.

    Returns:
        A dictionary containing the training and validation history
```

```

        (loss, accuracy, and learning rate) for each epoch.
    """
    # Set the random seed for reproducibility
    helper_utils.set_seed(10)

    # Define the loss function
    loss_fn = nn.CrossEntropyLoss()
    # Prepare the training and validation data loaders
    train_loader, val_loader = helper_utils.get_dataset_dataloaders(
        batch_size=batch_size
    )

    # Initialize a dictionary to store training and validation history
    history = {
        "train_loss": [],
        "train_acc": [],
        "val_loss": [],
        "val_acc": [],
        'lr': [],
    }

    # Initialize the progress bar for monitoring training
    pbar = helper_utils.NestedProgressBar(
        total_epochs=n_epochs,
        total_batches=len(train_loader),
        epoch_message_freq=5,
        mode="train",
    )

    # Loop through the specified number of epochs
    for epoch in range(n_epochs):

        # Update the progress bar for the current epoch
        pbar.update_epoch(epoch+1)

        # Train the model for one epoch
        train_loss, train_acc = helper_utils.train_epoch(model,
        train_loader, optimizer, loss_fn, device, pbar)
        # Evaluate the model on the validation set
        val_loss, val_acc = helper_utils.evaluate_epoch(model,
        val_loader, loss_fn, device)

        # Retrieve the current learning rate from the scheduler
        current_lr = scheduler.get_last_lr()[0]

        # Update the learning rate based on the scheduler type
        if isinstance(scheduler,
        optim.lr_scheduler.ReduceLROnPlateau):
            # For schedulers that monitor a metric, pass the metric to
            the step function

```

```

        scheduler.step(val_acc)
    else:
        # For other schedulers, call the step function without
arguments
        scheduler.step()

    # Log the training metrics for the current epoch, including
the learning rate
    pbar.maybe_log_epoch(epoch=epoch+1, message=f"At epoch
{epoch+1}: Training loss: {train_loss:.4f}, Training accuracy:
{train_acc:.4f}, LR: {current_lr:.6f}")

    # Log the validation metrics for the current epoch, including
the learning rate
    pbar.maybe_log_epoch(epoch=epoch+1, message=f"At epoch
{epoch+1}: Validation loss: {val_loss:.4f}, Validation accuracy:
{val_acc:.4f}")

    # Append the metrics for the current epoch to the history
dictionary
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)
    history['lr'].append(current_lr)

    # Close the progress bar upon completion of training
    pbar.close('Training complete!')
    # Return the collected training and validation history
    return history

```

In the following cells, you will run experiments using both `CosineAnnealingLR` and `ReduceLROnPlateau` schedulers. The training curves for each scheduler will be generated and compared to visualize their effects on model performance.

- **CosineAnnealingLR:**

This scheduler adjusts the learning rate following a cosine curve, gradually reducing it from the initial value to a minimum over a specified number of epochs.

Parameters used in this notebook are:

- `optimizer`: The optimizer to schedule.
- `T_max`: Number of epochs for one cycle of cosine annealing.
- `eta_min`: Minimum learning rate.

- **ReduceLROnPlateau:**

This scheduler monitors a metric (such as validation loss) and reduces the learning rate

by a factor if no improvement is seen for a set number of epochs.

Parameters used in this notebook:

- **optimizer**: The optimizer to schedule.
- **mode**: Whether to look for a decrease (**min**) or increase (**max**) in the monitored metric.
- **factor**: Factor by which the learning rate will be reduced.
- **patience**: Number of epochs with no improvement before reducing the learning rate.

For more details and additional parameters, see the [PyTorch documentation](#).

```
# CosineAnnealingLR
model = SimpleCNN().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.005)

scheduler_cosine = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=n_epochs, eta_min = 0.0002)

history_cosine = train_and_evaluate_with_scheduler(
    model, optimizer, scheduler_cosine, device, n_epochs=n_epochs,
    batch_size=batch_size
)

{"model_id": "764ed3088d86416fab60797996e209cd", "version_major": 2, "version_minor": 0}

{"model_id": "a6909c4ef3ce4c5daa0b0739d6f25074", "version_major": 2, "version_minor": 0}

{"model_id": "ad031789463549eeb1ca293a1d6ab7c7", "version_major": 2, "version_minor": 0}

{"model_id": "978d04721aff4e7d87950e65c2600ddb", "version_major": 2, "version_minor": 0}

{"model_id": "9914b8dfa9d941ee9e4fedc6dac4e7c2", "version_major": 2, "version_minor": 0}

{"model_id": "8e98828f53054682810cdfd96eb87671", "version_major": 2, "version_minor": 0}

{"model_id": "d22cda31d3e1479f8ad8e41b805158d5", "version_major": 2, "version_minor": 0}

At epoch 5: Training loss: 1.2436, Training accuracy: 0.5514, LR: 0.004703
At epoch 5: Validation loss: 1.2650, Validation accuracy: 0.5405
```

```
{"model_id": "388627c1fe3a493880aabcbe0bfff89df", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "28f9da21fd0d4b4e9735ce49c3f20278", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c12aa7ac611e4d7aadf424e1a8c639e6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c4ee9a38475741448f2cc81a5c2defd3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8dc6ea82f4be42a99e6cdb5d8aac1f40", "version_major": 2, "version_minor": 0}
```

At epoch 10: Training loss: 0.9252, Training accuracy: 0.6609, LR: 0.003622

At epoch 10: Validation loss: 1.2252, Validation accuracy: 0.5765

```
{"model_id": "68c1a2d1fe62445ca78f1b899f8c0942", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ae4b3cbb282c48019590419cbfef84d3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "53baac0751df48469c2f01258507894b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ce523b9140aa4d91a07d76b1ba9c97e5", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "94bfc7c1575a43d4a0df90eed6c17bbe", "version_major": 2, "version_minor": 0}
```

At epoch 15: Training loss: 0.6705, Training accuracy: 0.7536, LR: 0.002150

At epoch 15: Validation loss: 1.3185, Validation accuracy: 0.5955

```
{"model_id": "51d6101e0d65416cb645e804ec46a623", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "a8cddbaebe2844b59e56256105dc1f85", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "839a03bfe61c4858a917ad0d9092f4f7", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "d17845faead34fa28ef2c42b8804744d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1776c21a0c2d47e6a5feb7e31e390832", "version_major": 2, "version_minor": 0}
```

At epoch 20: Training loss: 0.5202, Training accuracy: 0.8053, LR: 0.000850

At epoch 20: Validation loss: 1.4080, Validation accuracy: 0.6075

```
{"model_id": "30b81f5c81a0475f99c444170d8ada76", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "598c8f68167741e888af8829deae4fb0", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "eb2e876cf15b4604b2f89830a3961773", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b723e8f8418445f581a578d1e06ba7ab", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "a440aa6796974da296ce464268346d8b", "version_major": 2, "version_minor": 0}
```

At epoch 25: Training loss: 0.4564, Training accuracy: 0.8284, LR: 0.000219

At epoch 25: Validation loss: 1.4661, Validation accuracy: 0.6065
Training complete!

ReduceLROnPlateau

```
model = SimpleCNN().to(device)
```

```
optimizer = optim.Adam(model.parameters(), lr=0.005)
```

```
scheduler_plateau = optim.lr_scheduler.ReduceLROnPlateau(optimizer,  
mode='max', factor=0.2, patience=3)
```

```
history_plateau = train_and_evaluate_with_scheduler(  
    model, optimizer, scheduler_plateau, device, n_epochs=n_epochs,  
    batch_size=batch_size  
)
```

```
{"model_id": "4c55eb7ee8f74542b67f0cd87ae5c06d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "af730db502fb4d5fa5a5a20c9e9aab87", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1e2852e0c6f84b47afd198fffe417357", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "080e32356c8741e692ea9c556e20f2a3", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b17c04b815d04b5190bf2cc387cba824", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "92b9c82108ef4472b538b06272db27de", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "06af0d19664d4575a23f41969420e3df", "version_major": 2, "version_minor": 0}
```

At epoch 5: Training loss: 1.2725, Training accuracy: 0.5433, LR: 0.005000

At epoch 5: Validation loss: 1.2231, Validation accuracy: 0.5645

```
{"model_id": "273a1859911b4b26b5c45ebb15195015", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "7074eef8c584a35a41d0029dbffb773", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "71914e1871c943ce8400af4892b9f516", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b55fa62dee1546858359fb8923475906", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "49c7a137654c45aea9ab277402fcacc3", "version_major": 2, "version_minor": 0}
```

At epoch 10: Training loss: 0.9621, Training accuracy: 0.6500, LR: 0.005000

At epoch 10: Validation loss: 1.2301, Validation accuracy: 0.5770

```
{"model_id": "c1a11531d5714e5a8d91c3a5d22ef4b7", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "44576257dd114961b5f4f97e5d6e95ca", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9b9748314d7d46838437c121a28c16f6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "40b69399771a4175a4354f6336fd8a22", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9abd20f6da054f0e930bc79dd01ba062", "version_major": 2, "version_minor": 0}
```

At epoch 15: Training loss: 0.7297, Training accuracy: 0.7304, LR: 0.005000

At epoch 15: Validation loss: 1.3620, Validation accuracy: 0.5825

```
{"model_id": "4149a72acf9440dd83f159d6379d719b", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e020f2ce0b2a42fabe834d94ee3ea813", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2f4da5c1a05243d5b137ddf0d2d763d4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "5ec1faf460644d688866c8bce11300ee", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "7ee346c97f714980b5384d2ac7e2404d", "version_major": 2, "version_minor": 0}
```

```
At epoch 20: Training loss: 0.4577, Training accuracy: 0.8255, LR: 0.001000
```

```
At epoch 20: Validation loss: 1.4694, Validation accuracy: 0.6050
```

```
{"model_id": "bf52f0f75819457faa8a1c25d6344794", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f2a8338ce71549af8a0001d7ba17fb81", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "0a2c7801f62e4757a2e56bb96344cee9", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "ac256208cfad43d1abd38f5c383c54d6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "8c36a9cea30a4579af0dac12793d9b77", "version_major": 2, "version_minor": 0}
```

```
At epoch 25: Training loss: 0.3850, Training accuracy: 0.8522, LR: 0.001000
```

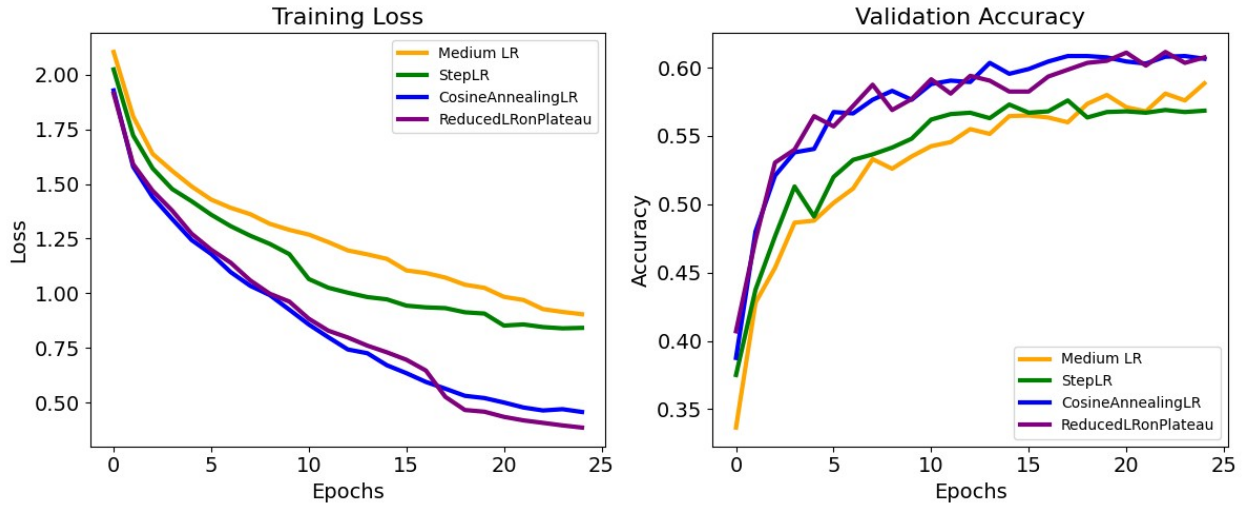
```
At epoch 25: Validation loss: 1.5742, Validation accuracy: 0.6075
```

```
Training complete!
```

```
labels = ['Medium LR', 'StepLR', 'CosineAnnealingLR',  
          'ReducedLROnPlateau']  
colors = ['orange', 'green', 'blue', 'purple']
```

```
training_curves_new = [history_constant, history_LR, history_cosine,  
                       history_plateau]
```

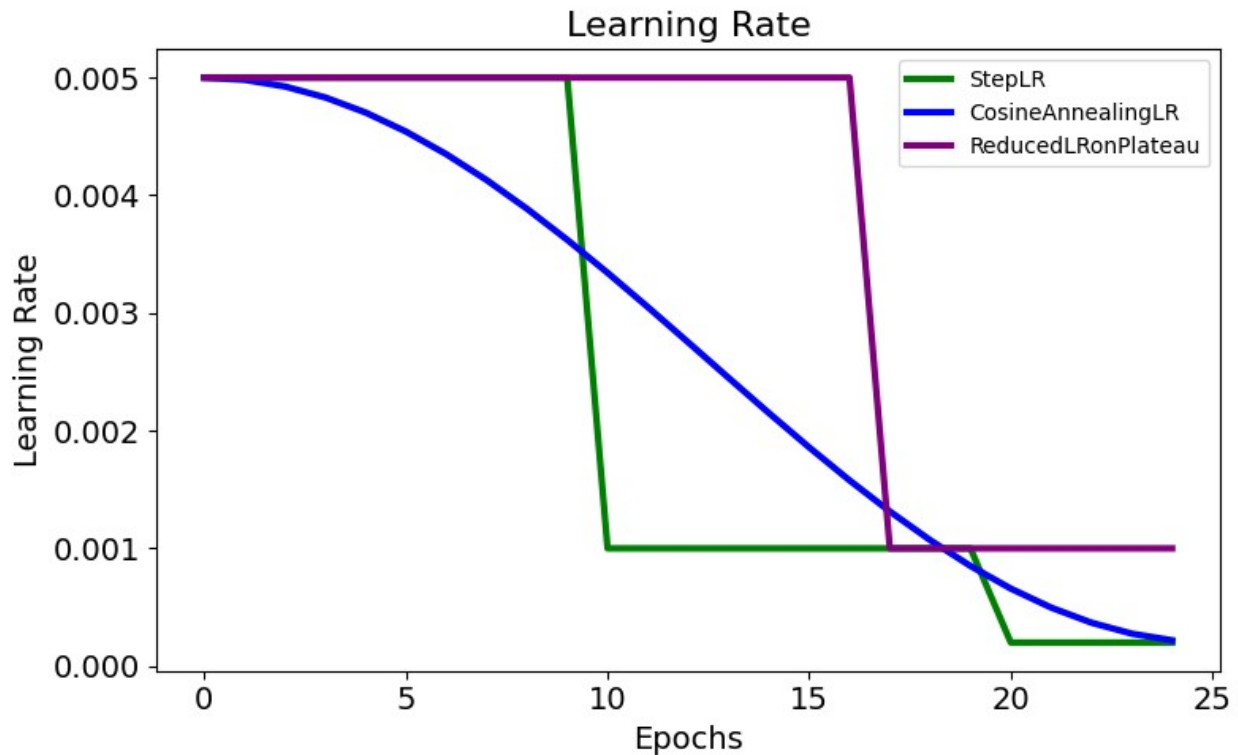
```
helper_utils.plot_learning_curves(colors, labels, training_curves_new)
```



Upon analyzing the validation accuracy, you see that both the `ReduceLROnPlateau` and `CosineAnnealingLR` schedulers yield improved results when you observe the last epoch. Keep in mind that each scheduler has tunable hyperparameters as well.

Below, you illustrate how the learning rate changes over the epochs for each scheduler. The `CosineAnnealingLR` scheduler reduces the learning rate according to a cosine decay pattern, while `ReduceLROnPlateau` adjusts it dynamically based on validation performance.

```
helper_utils.plot_learning_rates_curves(training_curves_new, colors, labels)
```



In summary:

- **StepLR** is effective for reducing the learning rate at fixed intervals in a stepwise manner.
- **CosineAnnealingLR** offers a smooth decay without sudden changes, which is beneficial for fine-tuning the model as training concludes. It's particularly advantageous for long training sessions where gradual adjustments can enhance convergence.
- **ReduceLRonPlateau** is useful when validation performance plateaus, as it adjusts the learning rate based on validation metrics, potentially improving generalization. It is responsive to the model's performance, allowing for dynamic adjustments based on training dynamics.

Conclusion

Congratulations on completing this in-depth exploration of PyTorch's learning rate schedulers! In this notebook, you examined how dynamically adjusting the learning rate can enhance training efficiency, speed up convergence, and improve generalization when compared to a constant learning rate. By utilizing schedulers and visualizing their effects on training and validation metrics, you observed how they contribute to improved model performance.

Additionally, you gained valuable experience in tuning scheduler hyperparameters to suit various training scenarios.